VLMo(2021, Microsoft)

> VLMo：灵活处理不同模态（2021，Microsoft）
> Unified **Vision-Language** Pre-Training with **Mixture-of-Modality-Experts**

1. VLMo 背景

为了将 VLP 任务中常见的这两种结构的优势拼在一起，作者提出了一种统一的视觉语言模型 VLMo。VLMo 相当于是一个**混合专家 Transformer 模型 (Mixture-of-Modality-Experts, MoME)**，预训练完成后，使用时既可以是双塔结构，实现高效地对图像和文本进行检索，又可以是单塔结构，对融合的图像和文本的特征进行建模和交互。

核心点：

1. MoME 架构
2. 不同模态的 Multi-Head Self Attention 层参数共享，只依靠 FFN 调整：在 Feed Foward （前向传播，FC）层，每个模态就会对应自己不同的 Expert
3. 分阶段的预训练策略：先训练单一模态，最后训练。

> 多模态的训练数据集不够多，但是在单模态里，Vision 或者 NLP 可用的数据很多，所以采用分阶段的预训练策略来训练，效果会提升很多。

2. VLMo 模型结构

2.1 输入表示

图像的表示：

图像的表示与 ViT 的做法一样。VLMo 多了一个类型编码 $V_{type} \in R^D$ 。

最终得到图像表示：$H_0^v = [v_{I_CLS}, V_{v_i^p}, \ldots, V_{v_N^p}] + V_{pos} + V_{type}$ ，式中 $H_0^v \in R^{(N+1)\times D}$ ，投影为 $V \in R^{(P^2C)\times D}$ ，其中 v_{I_CLS} 是表示图像全局特征的标记。

文本的表示：

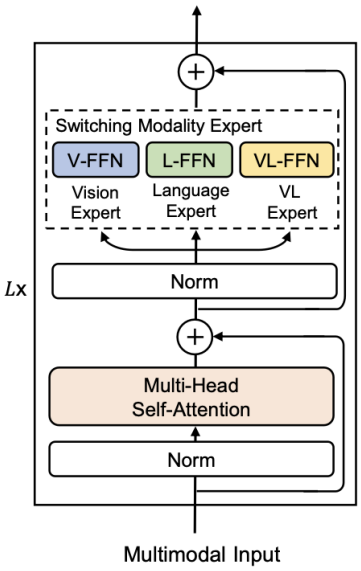
文本的表示与Bert一样，都是采用 BPE 进行分词。VLMo 也是多了一个类型编码 $T_{type} \in R^D$。

最终得到文本表示：$H_0^w = [w_{T_CLS}, w_i, \ldots, w_M, w_{T_SEP}] + T_{pos} + T_{type}$ ，式中 $H_0^w \in R^{(M+2)\times D}$ ，投影为 $W \in R^{(P^2C)\times D}$ ，其中 M 是 tokenized 词根长度，w_{T_CLS} 是表示文本全局特征的标记，w_{T_SEP} 是表示不同句子的区分。

图像-文本对的表示：

将图像和文本输入向量拼接在一起形成文本和图像的表示：$H_0^{vl} = [H_0^w; H_0^v]$

2.2 MoME 模型架构



MoME Transformer with Shared Parameters

VLMo 是一个混合专家 Transformer 模型（MoME）。依旧遵循 Transformer 的一般设计思路，但是做出了一定的改动。

MoME Transformer 也是先一个 Multi-Head Self-Attention（MHSA）的子模块，再串联一个 Feed Forward Network（FFN）的子模块。但是和正常 Transformer 不同的是，引入了模态专家的混合，以替代标准 Transformer 的前馈网络。在 MoME Transformer 中的 FFN 子模块是由 3 个并行、独立的 FFN 并联得到。这 3 个 FFN 分别是 Vision-FFN（V-FFN，视觉专家），Language-FFN（L-FFN，文本专家）和 Vision-Language-FFN（VL-FFN，视觉文本专家），针对不同的模态输入对应不同的专家，所以是 **switching modality expert**。不同 FFN 的结构没有任何不同，都是：MLP + GELU + LayerNorm。在代码中用`modality_type='image'\'text'\'vl'`参数来选择进入到不同的模块中。

⚠ 不同模态的 **Multi-Head Self-Attention（MHSA）**共享权重，但是不同模态的 FFN 独立学习，捕获不同模态下的信息。
即，不论是图像、文本还是图像-文本信号，输入任何的 token 序列，self-attention 的模型权重都是一样的，这个也是 transformer 架构的优势，或者说是多模态学习接下来的趋势。
有一些工作证明，同样的 self-attention 权重可以用来做不同的图像文本音频视频任务，不需要重新去训练自注意力参数。

MoME Transformer 的主要组成结构：

1. 共享的 MHSA；
2. 适配不同任务的 FFN 模块（expert 专家）。

流程：
给定一个输入数据，我们首先将它输入到共享的多头自注意力层，然后再根据不同的输入数据类型，MoME Transformer 选择切换不同的专家进行训练。

假设 MoME Transformer 的前一层的输出（当前层的输入）表示为 H_{l-1}，那么 MoME Transformer 的计算方式可以简化为：

$$H_l' = MSA(LN(H_{l-1})) + H_{l-1},\ H_l = MoME\text{-}FFN(LN(H_l')) + H_l'$$

VLMo 的灵活性：

- 如果输入是**纯图像或者纯文本**，就使用视觉专家和文本专家分别对输入的图像和文本进行编码；
- 如果输入由**多模态的向量组成**，比如是**图像和文本的拼接**，就先使用视觉专家和文本专家分别对输入的图像和文本进行编码，再使用多模态专家进行联合地交互。

VLMo 根据需求可以灵活地使用不同的专家模型，不论是架构层面还是训练层面都很灵活。

3. VLMo 预训练

为了利用大规模的纯图像的数据集和纯文本的语料库，设计了一种**分阶段的预训练方法**：对纯图像数据的训练，对纯文本数据的训练，以及混合模态数据的训练。

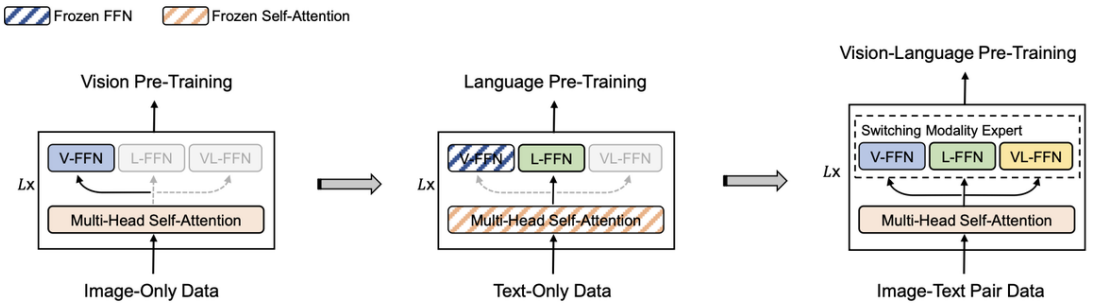
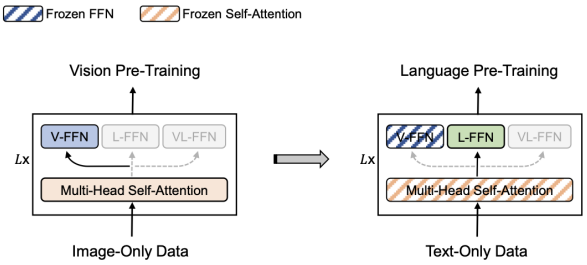


Figure 2: Stagewise pre-training using image-only and text-only corpora. We first pretrain the vision expert (V-FFN) and self-attention module on large-scale image-only data as in BEIT [2]. Then the parameters of vision expert and self-attention module are frozen, and we train the language expert (L-FFN) by masked language modeling on large amounts of text-only data. Finally, we train the whole model with vision-language pre-training.

3.1 单模态

单模态的数据训练包括**纯文本数据的训练**和**纯图像数据的训练**。



3.1.1 Image-Only

1. 阶段一：纯图像数据：
   - VLMo 采用了 BEiT 的图像完形填空 MIM （掩码图像模型 Maksed Image Model）进行预训练。
   - 只使用 V-FFN 专家模型，更新 MHSA 和 V-FFN 的参数

3.1.2 Text-Only

2. 阶段二：纯文本数据的训练：
   - VLMo 采用了 BERT 中的完形填空 MLM（掩码语言模型 Masked Language Model）进行模型的预训练。

- 只使用 L-FFN 专家模型，**冻住 MHSA 和 V-FFN 的参数，更新 L-FFN 参数，意思就是在这一步中使用前一步中训练好的 MHSA 模型参数。**
- ⚠ 在视觉数据上训练的自注意力模型（MHSA），可以直接用于文本建模，不需要 fine-tune，就可能表现很好。但是如果我现在文本数据上训练 MHSA，然后直接用于图像建模，结果就不是很好。

补充 MIM

介绍一下**Maksed Image Model，MIM**：

BEiT 是图像界的 BERT 模型，所以也遵循着和 BERT 相似的自监督训练方法 MLM 。BEiT 使用了类似 BERT 的自监督训练方式：Masked Image Modeling，MLM，即：

> 给 BEIT 看很多的图片，随机盖住一些图片的 patches，让 BEIT 模型预测盖住的 patches 是什么，不断计算预测的 patches 与真实的 patches 之间的差异，利用它作为 loss 进行反向传播更新参数，来达到 Self-Supervised Learning 的效果。

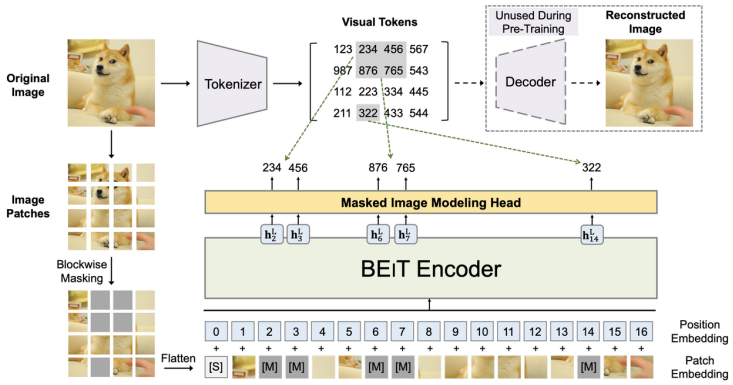
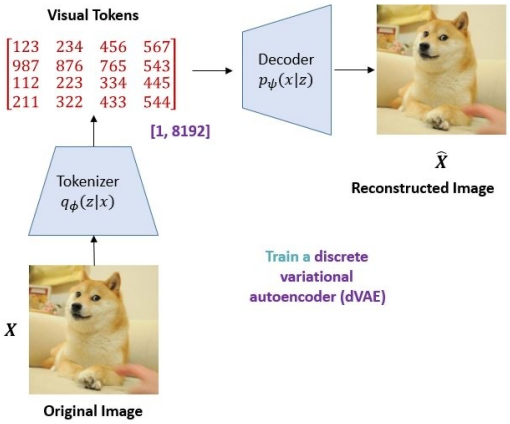


Figure 1: Overview of BEIT pre-training. Before pre-training, we learn an "image tokenizer" via autoencoding-style reconstruction, where an image is tokenized into discrete visual tokens according to the learned vocabulary. During pre-training, each image has two views, i.e., image patches, and visual tokens. We randomly mask some proportion of image patches (gray patches in the figure) and replace them with a special mask embedding `[M]`. Then the patches are fed to a backbone vision Transformer. The pre-training task aims at predicting the visual tokens of the *original* image based on the encoding vectors of the *corrupted* image.

具体做法是（引用）：

1. 给定输入图片 x。
2. 通过 patch embedding 等类似的方法将 x 变成 N 个图片 patches $\{x_i^p\}_{i=1}^N$ 。
3. 使用经过 dVAE 训练的 tokenizer 将上一步变成 N 个 visual tokens$\{z_i\}_{i=1}^N$。

> tokenizer：（同样也是真实 patches 对应的 visual token 的获取方式）
> 作者把一张 224×224 的输入图片通过 Tokenizer 变成了 14×14 个 visual token，每个 visual token 是一个位于 [1, 8192] 之间的数值。就像有个 image 的词汇表一样，这个词汇表里面有 8192 个词，每个 16×16 的 image patch 会经过 Tokenizer 映射成 $|V|$ 里面的一个词。



（图片引用自：https://www.cvmart.net/community/detail/4980）

4. 随机盖住 40% 的图片 patches，盖住的位置可以表示为 $M \in \{1, \ldots, N\}^{0.4N}$ 。

随机遮盖 40% 的图片 patches 的方式：Blockwise Masking

Algorithm 1 Blockwise Masking

Input: $N(= h \times w)$ image patches
Output: Masked positions $\mathcal{M}$
$\mathcal{M} \leftarrow \{\}$
repeat
  $s \leftarrow \text{Rand}(16, 0.4N - |\mathcal{M}|)$ ▷ *Block size*
  $r \leftarrow \text{Rand}(0.3, \frac{1}{0.3})$ ▷ *Aspect ratio of block*
  $a \leftarrow \sqrt{s \cdot r}; b \leftarrow \sqrt{s/r}$
  $t \leftarrow \text{Rand}(0, h - a)$; $l \leftarrow \text{Rand}(0, w - b)$
  $\mathcal{M} \leftarrow \mathcal{M} \bigcup \{(i, j) : i \in [t, t + a), j \in [l, l + b)\}$
until $|\mathcal{M}| > 0.4N$ ▷ *Masking ratio is 40%*
return $\mathcal{M}$

A. 初始化，输入：

- 一张图像被划分成 `h × w` 个 patches，每个 patch 的大小为 `16 × 16`。
- 设 `N = h × w`，表示总的 patches 数量。
- 目标是 mask `0.4N` 个 patches。
- 定义一个空集合 $\mathcal{M}$，用于存储被 mask 的 patches。

B. 进入循环，直到被 mask 的 patches 数量达到 40%
(循环步骤)

1. 随机确定一个 block 的大小 `s`（block 内包含的 patches 数量），设定范围
2. 随机选择 block 的纵横比 `r`
3. 计算 block 的宽 `b` 和高 `a`，确保面积合理
4. 在 `h × w` 网格中随机选取 block 的位置，**确保 block 不会超出图像边界。**
5. 遮盖 block 内所有的 patches
6. 检查是否达到了 40%

C. 返回最终的遮盖区域

- 最终输出 $\mathcal{M}$，其中包含所有被 mask 的 patches 坐标。
- 这 40% 的 patches 作为 BEiT 预训练的 masked patches，模型需要预测它们的视觉 token。

⚠：BEiT 的 blockwise masking 以 block（矩形区域）为单位遮挡完整的 patches，最终遮盖 40% 的 patches，而不会部分遮盖某个 patch。

效果例子：

假设我们有 `14×14` patches 的网格（1 个 patch 大小为`16×16`），下面是 `blockwise masking` 可能的效果：

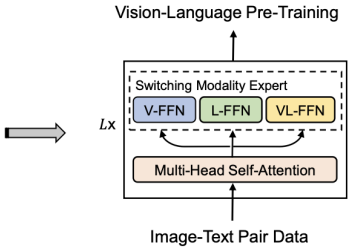
```
. . X X X . . . . . . . . .
. . X X X . . . . . . . . .
. . X X X . . X X X X . . .
. . . . . . . X X X X . . .
. . . . . . . X X X X . . .
. X X . . . . X X X X . . .
. X X . . . . . . . . . . .
. X X . . . . . . . . . . .
. . . . . . . . . X X X X
. . . . . . . . . X X X X
. . . . . . . . . X X X X
. . . . . . . . . . . . . .
. . . . . . . . . . . . . .
. . . . . . . . . . . . . .
```

- `X` 代表被 mask 的 patches。
- 你可以看到 **遮盖的 block 是多个 patches 组成的矩形区域。**
- 没有 patch 被 **部分遮盖**，要么完全 `X`，要么完全 `.`。

5. 把盖住的这 40% 的图片 patches 替换成可学习的编码 $e_{[m]} \in R^D$ 。
6. 现在这个输入的图片 patches 就可以表示成原本的 patch 和遮盖后 patch 的并集：$x^M = \{x_i^p : i \notin M\}_{i=1}^N \cup \{e_{[M]} : i \in M\}_{i=1}^N$
7. 把这个 x^M 通过 L 层的 BEiT Encode，得到表示输入 image patches 的编码表示。
8. 将【盖住的位置的输出】去通过一个分类器，预测盖住的这个 patch 相应的 visual token，类似 BERT 中的流程。
9. BERT 的训练目标是最小化计算预测的 token 与真实的 token 之间的差异，所以 **BEiT 的目标也是最小化计算预测的 token 与真实的 token 之间的差异。**但其实并不完全一致，在 BEiT 里面，假设盖住第 i 个 patch，毫无疑问它对应的 visual token 应该是 z_i，这时候我会希望 Encoder 输出的第 i 个位置的东西通过分类器之后是 z_i 的概率最大。（简单一点，就是**希望通过分类器之后得到的预测的 visual token 与真实 patches 对应的 visual token 越接近越好**）

以上就是纯图像预训练的方法**MIM**。

3.2 多模态



在第一阶段 Image-Only 和第二阶段 Text-Only 训练结束后，视觉和文本专家的参数都得到了很好的初始化，进入第三阶段 Image-Text Pair 多模态的训练，在这一步中会更新所有模型的参数，泛化性能更好。使用经典老三样**Image-Text Contrast（ITC）**、**Masked Language Modeling（MLM）**和**Image-Text Matching（ITM）**三个任务进行训练。

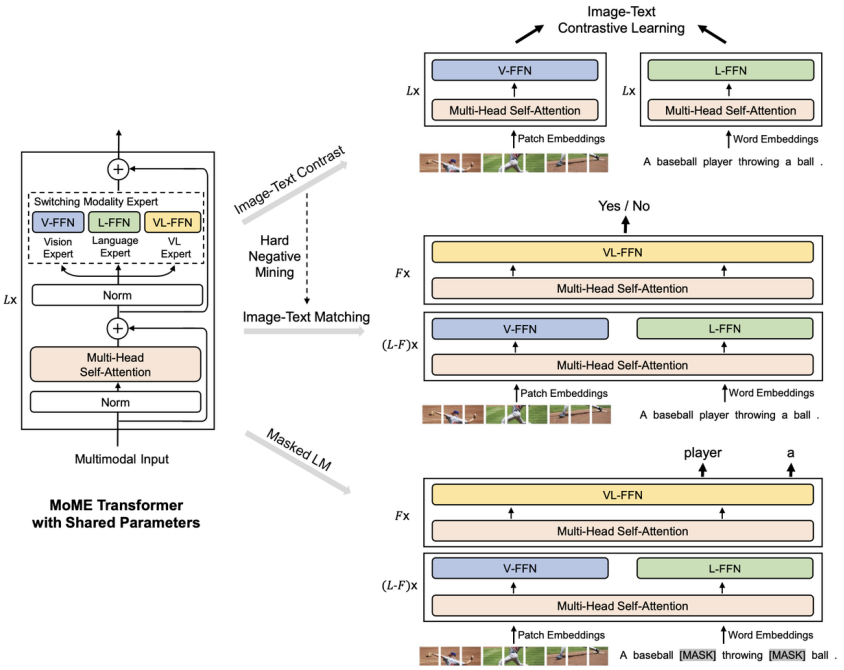


Figure 1: Overview of VLMo pre-training. We introduce mixture-of-modality-experts (MoME) Transformer to encode different modality input by modality-specific experts. The model parameters are shared across image-text contrastive learning, masked language modeling, and image-text matching pre-training tasks. During fine-tuning, the flexible modeling enables us to use VLMo as either a dual encoder (i.e., separately encode images and text for retrieval tasks) or a fusion encoder (i.e., jointly encode image-text pairs for better interaction across modalities).

L 层 transformer 分类两部分，前 (L-F) 层和后 F 层。前 (L-F) 层仅使用 V-FFN 和 L-FFN（模型对图片和文本分别做训练），后 F 层的使用情况视任务而定（在 ITM 和 MLM 中，在 F 层在 VL-FFN 训练）。

⚠ VLMo 中，未用到 cross attention

⏏：如果模型结构用的是 Vision Transformer Base，一共 12 层，那么前 10 层用 V-FFN 和 L-FFN，后 F=2 层用 VL-FFN，即后面只有 2 层 transformer block 做模态之间的融合。

- **训练图文对比学习任务 ITC：ITC 的设计目的是让不同模态的在特征空间对齐。**
  - ITC 的训练遵循 CLIP 的做法，图像和文本使用各自的专家模型（⚠ **这里 MHSA 是共享的参数**），比如图像端就是一个 12 层的 ViT，文本端就是一个 12 层的 Bert。
  - 实验中是将 image embedding 和 text embedding 进行拼接一块输入，attention 中使用一个 mask 矩阵防止两个模态相互看到，特征空间使用对比学习损失（交叉熵）。
- **训练完形填空任务 MLM：**
  - 模型变成有 fusion ecoder 的形式（增加了 VL-FFN）。在网络的前 (L-F) 层，不同模态拼接在一次，通过共享的 MHSA ，再拆分成不同模态的 embedding 通过各自的专家模型即可。在网络的后 F 层，通过共享 MHSA 后在通过 VL-FFN 模型。
  - 遵循 Bert 的做法，随机 mask 掉 15% 的文本 token，然后以图文对的方式输入到 F 层的 MHSA 中（这里的 MHSA 与之前和后面的 attention 都是共享权重的），训练 VLMo 模型使之从” 未被 mask 掉的文本” 和” 视觉信息” 中预测盖住的词是什么。使用分类器得到 masked token 的输出分布，交叉熵损失训练。
- **训练图文匹配任务 ITM：ITM 的设计目的是预测图文对是否匹配，二分类任务。**
  - 提出了全局难负样本 hard negatives 挖掘，并从所有 GPU (全局计算) 收集到的更多训练示例中的 hard negative 图像-文本对进行采样，区别于 ALBEF 的单个 gpu。

总结三个阶段的预训练：

1 视觉预训练：
Image-Only data
在纯图像数据集上面预训练视觉专家模型，方案就是 MIM 的做法。文本专家模型和视觉文本专家模型的 FFN 权重冻结，共享的 Self-Attention 参数更新。

2 文本预训练：
Text-Only data
在纯文本数据集上面预训练文本专家模型，方案就是 MLM 的做法。视觉专家模型和视觉文本专家模型的 FFN 权重冻结，共享的 Self-Attention 参数也冻结，注意这里的 Self-Attention 的参数还是视觉数据集上面训练出来的了。

3 多模态预训练：
Image-Text Pair data
在多模态数据集上面预训练文本专家模型，方案就是 ITC、MLM、ITM 三个目标函数。所有参数都开放更新。

1. MoME Transformer 这种灵活性模型架构设计的好处：
   - 做 **ITC** 时，VLMo 和 CLIP 很像，可以让视觉专家 V-FFN 输入视觉信息，让文本专家 L-FFN 输入文本信息，把得到的输出做对比学习即可。
   - 做 **ITM** 时，可以在前几个 Block 让视觉专家 V-FFN 输入视觉信息，让文本专家 L-FFN 输入文本信息，在后几个 Block 把信息给视觉文本专家，然后做二分类即可。
   - 做 **MLM** 时，可以在前几个 Block 让视觉专家 V-FFN 输入视觉信息，让文本专家 L-FFN 输入文本信息，在后几个 Block 把信息给视觉文本专家，然后做预测即可。

1. ITC、ITM、MLM、MIM 分别在哪里使用，一共用了几次？
   ITC、ITM：第三阶段 Image-Text Pairs 多模态训练中，1 次；
   MLM：第二阶段 Text-Only 文本训练和第三阶段 Image-Text Pairs 多模态训练中，2 次；
   MIM：第一阶段 Image-Only 图片训练中，1 次。

1. 为什么在视觉上训练好的 MHSA，在文本任务中表现也很好？
   视觉数据的训练方式对 MHSA 具有更强的泛化性，原因：
   ☑ **(1) 视觉 MHSA 需要学习复杂的 2D 关系，能力更泛化**
   - 视觉 Transformer（ViT）将图像划分成 Patch 序列，每个 Patch 是一个 Token（例如 16x16 像素）。
   - 视觉中的 **物体、边缘、层次结构、局部特征、远程依赖关系**都需要被 MHSA 捕捉。
   - **这种能力可以自然地迁移到文本数据**，因为文本也依赖 **上下文关系、长距离依赖等特征。**
   - 由于视觉 Token 是一个 2D 结构，MHSA 被训练成**更泛化的关系建模方法**，因此直接迁移到文本时，它仍然可以有效建模文本 token 之间的关系。

   ☑ **(2) 视觉训练让 MHSA 习得更强的注意力分布**
   - 在 **视觉任务中**，MHSA 需要学习哪些 Patch 之间存在**重要的联系**（例如，在目标检测或图像分类任务中）。
   - 这种能力在文本数据中**同样适用**，因为 NLP 任务也需要学习**哪些词语之间有重要联系**（如句法结构、语义关系）。
   - 所以，**在视觉上预训练的 MHSA 已经学会了如何合理地分配注意力，因此可以无缝适配到文本任务。**

☑ **(3) 视觉训练的 MHSA 具有更好的归纳偏置（Inductive Bias）**

- 视觉任务的 MHSA 需要学习到**局部感受野（locality）、层级结构（hierarchical structure）、长程依赖（long-range dependencies）**，而这些结构在文本数据中同样存在。
- 由于视觉中的注意力结构比文本更加丰富，MHSA 在视觉上训练时，会更全面地学习如何建模不同类型的关系。
- 所以，**视觉训练的 MHSA 在文本任务上依然适用，而反过来却不行。**

2. 为什么先在文本上训练 MHSA，然后迁移到视觉任务效果不佳?

虽然 MHSA 是通用的，但文本和视觉数据的结构存在一些**关键性差异**，导致 **文本训练的 MHSA 在视觉任务上效果不好。**

❌ **(1) 文本 MHSA 主要关注局部关系，泛化能力较差**

- **文本数据是线性的 1D 序列**，主要关注 **短距离依赖（short-range dependency）。**
- **视觉数据是 2D 结构**，需要建模 **多尺度（multi-scale）、空间结构（spatial structure）和远程依赖（long-range dependency）。**
- **如果 MHSA 先在文本数据上训练，它更容易过拟合 1D 依赖模式，导致它在视觉数据上泛化能力变差。**

❌ **(2) 文本 MHSA 的注意力机制无法处理复杂空间关系**

- 在 NLP 任务中，注意力主要用于学习**词序、句法结构**，但视觉任务需要更强的**空间建模能力。**
- **文本数据中没有物理空间关系，视觉数据有明显的空间结构（2D 关系）。**
- **如果 MHSA 先在文本任务上训练，它在处理 2D 视觉数据时会缺乏必要的 inductive bias，导致性能下降。**

❌ **(3) 文本数据的 token 表示方式和视觉不同**

- **文本 Token 是离散的（discrete），每个词有固定的 embedding。**
- **视觉 Patch 是连续的（continuous），需要额外的 Positional Embedding 和归一化。**
- **如果 MHSA 先在文本任务上训练，它的参数可能更适应离散数据，而不适用于视觉任务。**

1. VLMo 为什么没有用 cross attention?

核心原因：在 VLMo 中，视觉 token 和文本 token 都通过同一个多头自注意力（MHSA）来进行交互，其参数（如 W_Q，W_K，W_V）**在所有模态上共享。**

(1) 传统多模态模型：

- 视觉和文本有各自单独的自注意力（例如 Vision Encoder + Language Encoder）。
- 为了让视觉特征与文本特征融合，需要额外的 cross attention，通过一组专门的注意力参数来“跨模态”匹配 Q（来自文本）和 K、V（来自视觉）或反之。

(2) VLMo 的做法：

- 只有一个“统一”Transformer，把视觉 token 与文本 token 拼在一起，当成一个长序列来做**自注意力。**
- 这个自注意力层的 W_Q，W_K，W_V **等参数在所有 token 之间共享**，模型可以根据输入的类型（视觉或文本 token）动态学到如何互相注意。
- 因此，模型不需要再显式设计“视觉对文本”或“文本对视觉”的 cross attention，因为**共享的自注意力层就能自然地完成跨模态信息的交互。**

核心代码，不同专家的实现：

代码块

```
""" Vision Transformer (ViT) in PyTorch
实现Vision Transformer的PyTorch版本，参考论文：
'An Image Is Worth 16 x 16 Words: Transformers for Image Recognition at Scale'
官方Jax代码：https://github.com/google-research/vision_transformer

主要贡献/改动：
1. 添加了多模态处理能力（图像+文本）
2. 支持分阶段训练（单模态/跨模态）
3. 实现模态特定的前馈网络（FFN）
"""

......

class Block(nn.Module):
    """Transformer块（包含注意力+模态特定FFN）"""
    def __init__(
        self,
        dim,                    # 输入维度
        num_heads,              # 注意力头数
        mlp_ratio=4.0,          # MLP扩展比例
        qkv_bias=False,         # 是否在QKV中使用偏置
        qk_scale=None,          # 缩放因子
        drop=0.0,               # 基础dropout率
        attn_drop=0.0,          # 注意力dropout率
        drop_path=0.0,          # 随机深度率
        act_layer=nn.GELU,      # 激活函数
        norm_layer=nn.LayerNorm, # 归一化层
        with_vlffn=False,       # 是否包含跨模态FFN
        layer_scale_init_values=0.1, # 层缩放初始化值
        max_text_len=40,        # 最大文本长度（用于特征分割）
    ):
        super().__init__()
        # 归一化层
        self.norm1 = norm_layer(dim)
        # 注意力模块
        self.attn = Attention(
            dim, num_heads=num_heads, qkv_bias=qkv_bias,
            qk_scale=qk_scale, attn_drop=attn_drop, proj_drop=drop
        )
        # 随机路径丢弃
        self.drop_path = DropPath(drop_path) if drop_path > 0.0 else nn.Identity()

        # 模态特定归一化层
        self.norm2_text = norm_layer(dim)
        self.norm2_imag = norm_layer(dim)

        # MLP参数计算
        mlp_hidden_dim = int(dim * mlp_ratio)

        # 文本模态FFN
        self.mlp_text = Mlp(
            in_features=dim,
            hidden_features=mlp_hidden_dim,
            act_layer=act_layer,
            drop=drop,
        )
```

```
        # 图像模态FFN
        self.mlp_imag = Mlp(
            in_features=dim,
            hidden_features=mlp_hidden_dim,
            act_layer=act_layer,
            drop=drop,
        )
        # 跨模态FFN（可选）
        self.mlp_vl = None
        if with_vlffn:
            self.mlp_vl = Mlp(
                in_features=dim,
                hidden_features=mlp_hidden_dim,
                act_layer=act_layer,
                drop=drop,
            )
            self.norm2_vl = norm_layer(dim)

        # 层缩放参数（类似ConvNeXt设计）
        self.gamma_1 = nn.Parameter(layer_scale_init_values * torch.ones(dim)) \
            if layer_scale_init_values else 1.0
        self.gamma_2 = nn.Parameter(layer_scale_init_values * torch.ones(dim)) \
            if layer_scale_init_values else 1.0

        self.max_text_len = max_text_len  # 最大文本长度（用于分割特征）

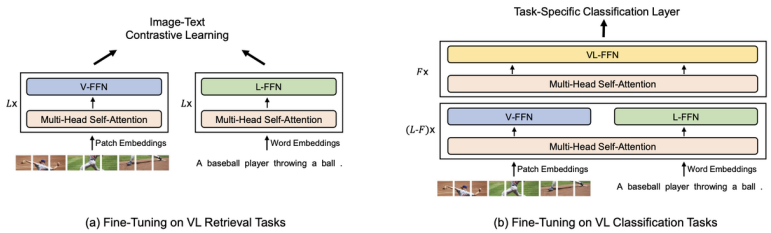
    def forward(self, x, mask=None, modality_type=None, relative_position_bias=None):
        # 残差连接1：注意力部分
        x = x + self.drop_path(
            self.gamma_1 * self.attn(  # 层缩放
                self.norm1(x),         # 先归一化
                mask=mask,
                relative_position_bias=relative_position_bias
            )
        )

        # 残差连接2：模态特定FFN
        if modality_type == "image":  # 图像模态
            x = x + self.drop_path(
                self.gamma_2 * self.mlp_imag(self.norm2_imag(x))
            )
        elif modality_type == "text":  # 文本模态
            x = x + self.drop_path(
                self.gamma_2 * self.mlp_text(self.norm2_text(x))
            )
        else:  # 跨模态或混合模态
            if self.mlp_vl is None:
                # 分割文本和图像特征
                x_text = x[:, :self.max_text_len]
                x_imag = x[:, self.max_text_len:]
                # 分别处理文本和图像
                x_text = x_text + self.drop_path(
                    self.gamma_2 * self.mlp_text(self.norm2_text(x_text)))
                x_imag = x_imag + self.drop_path(
                    self.gamma_2 * self.mlp_imag(self.norm2_imag(x_imag)))
                x = torch.cat([x_text, x_imag], dim=1)
            else:
                # 使用跨模态FFN
```

```
            x = x + self.drop_path(
                self.gamma_2 * self.mlp_vl(self.norm2_vl(x)))
        return x
```

4. VLMo 微调

可以通过对 VLMo 进行微调以适应不同的下游任务，在论文中有两种下游任务：



(a) Fine-Tuning on VL Retrieval Tasks　　(b) Fine-Tuning on VL Classification Tasks

Figure 3: Fine-tuning VLMo on vision-language retrieval and classification tasks. The model can be fine-tuned as a dual encoder to separately encode image and text for retrieval tasks. VLMo can also be used as a fusion encoder to handle interaction of image-text pairs for classification tasks.

- 视觉语言检索（上图 (a)）：
  - 双塔结构
  - 在微调期间，使用图像-文本对比损失来优化。在推理过程中，作者计算所有图像和文本的表示，然后使用点积来获得所有可能的图像-文本对的图像到文本和文本到图像相似度分数。VLMo 的这个做法可以实现更快的推理速度。
- 视觉语言分类（上图 (b)），包括视觉问答 VQA 和视觉推理 NLVR 等：
  - 单塔结构
  - 先通过视觉专家和文本专家建模各自特征，再联合送入多模态专家完成更复杂的交互，完成分类任务。VLMo 使用 [T_CLS] token 的最终编码向量作为图像-文本对的表示，并加上分类头预测类别。

5. 总结

VLMo 的 MoME 设计，兼顾了“统一”与“专精”的平衡，“灵活”是 VLMo 最突出的亮点：

1. **统一的自注意力避免了繁琐的跨模态模块。**
2. **Mixture-of-Experts 让视觉、语言、跨模态等不同需求都能通过并行专家来独立学习，还能在推理时自动调度“最优专家组合”。**
3. 模型可**一体化处理单/多模态任务**，提升了迁移学习和下游适配的效率。

VLMO 还用大量的实验证明 Self-Attention 层确实是可以共享参数的，它跟模态没什么关系。

发布 VLMo 微软这个团队在那几年，真的是出了很多厉害的工作，包括 BEiT v1、v2、v3，还有 LayoutLM v1、v2、v3，还有做语音的，做视频的，可以说是多模态领域里非常 solid 的一个组。所以大家如果想做多模态学习，任何一个领域的多模态学习，都可以去看一下他们组发的论文，应该或多或少都会有一些联系的。

BEiT 系列工作的发展历程：

1）BEiT V1：2021 年 6 月份发布

2）VLMo：2021 年 11 月份发布

3）VL-BEiT：22 年 6 月份发布，作者团队同时用 Mask Modeling 做 Vision 和 Language。

4）BEiT V2：22 年 8 月份发布，BEiT V2 是 BEIT V1 的一个升级版，它还是做的视觉的 Dataset。

5）BEiT V3：22 年 8 月份发布，BEiT V3 就是之前所有工作的一个集大成版本，一个多模态的网络同时做单模态和多模态任务。